FPT UNIVERSITY

PROJECT REPORT

THIẾT KẾ VÀ KIỂM CHỨNG BỘ ĐIỀU CHẾ AM (DSB-FC) TRÊN FPGA BẰNG VERILOG RTL

Team: 3

Class: IC2010

Team members: CAO HỮU AN

PHẠM MINH TÙNG

MAI THẾ TÂM

LÊ THẾ TRƯỜNG

PHAN ANH MINH

TRẦN MINH TRIẾT

Lecturer: Dr. Phạm Thế Vinh

HO CHI MINH CITY, NOVEMBER 2026

MỤC LỤC

1. GIỚI THIỆU TỔNG QUAN (INTRODUCTION) ..... - 2 -
2. CƠ SỞ LÝ THUYẾT VÀ TÍNH TOÁN ĐIỂM CỐ ĐỊNH (THEORETICAL & FIXED-POINT CONTRACT) ..... - 2 -
2.1. Biểu thức AM DSB-FC và Chuyển đổi Số nguyên ..... - 2 -
3. KIẾN TRÚC PHẦN CỨNG RTL (HARDWARE ARCHITECTURE) ..... - 4 -
3.1. Sơ đồ khối hệ thống (Block Diagram) ..... - 4 -
3.2. Cấu trúc và Chức năng Module (Module Index) ..... - 4 -
4. QUY TRÌNH KIỂM CHỨNG TESTBENCH (VERIFICATION) ..... - 5 -
4.1. Self-Checking Testbench (tb_am_top.v) ..... - 5 -
4.2. Kiểm chứng So sánh với Golden Model Python (RTL vs Reference) ..... - 5 -
4.3. Kiểm thử Đơn vị Python (Python Unit Tests) ..... - 6 -
5. KẾT QUẢ TỔNG HỢP VÀ ĐÁNH GIÁ TÀI NGUYÊN (SYNTHESIS RESULTS & ANALYSIS) - 7 -
5.1. Khảo sát Đánh giá Giả thuyết (Hypotheses Under Test) ..... - 7 -
5.2. Khung Bảng Kết quả Tổng hợp Vivado Batch Sweep ..... - 7 -
6. HẠN CHẾ CỦA THIẾT KẾ (LIMITATIONS) ..... - 7 -
7. ĐÁNH GIÁ VÀ SO SÁNH HIỆU NĂNG (EVALUATION & COMPARISON) ..... - 8 -
7.1. So sánh Kiến trúc Full-Wave ROM và Quarter-Wave ROM ..... - 8 -
7.2. So sánh Phương án ROM-based NCO với Phương án Xấp xỉ Thuật toán (CORDIC / Đa thức) ..... - 8 -

1. GIỚI THIỆU TỔNG QUAN (INTRODUCTION)

Điều chế biên độ (Amplitude Modulation - AM) dạng sóng mang đầy đủ hai dải bên (Double Sideband Full Carrier - DSB-FC) là một trong những kỹ thuật cơ bản nhất trong xử lý tín hiệu truyền thông. Dự án này thực hiện hiện thực hóa toàn bộ luồng xử lý phần cứng (RTL Design) cho bộ điều chế AM bằng ngôn ngữ Verilog HDL.

Các thông số thiết kế chính (Design Specifications):

- **Tần số xung nhịp hệ thống (f_clk):** 100 MHz.
- **Độ rộng luồng dữ liệu (Datapath Width):** 12-bit signed cho mẫu sóng sin, 16-bit signed cho đầu ra am_out.
- **Bộ tích lũy pha (Phase Accumulator):** 32-bit cho độ phân giải tần số cao.
- **Độ trễ hệ thống (Pipeline Latency):** Đúng 3 chu kỳ clock từ đầu vào đến am_out.
- **Kiến trúc ROM:** Hỗ trợ 2 phương pháp Lookup (Full-wave ROM và Quarter-wave ROM) để so sánh tối ưu tài nguyên.
- **Thiết bị mục tiêu tổng hợp (Target Device):** Xilinx Artix-7 (xc7a35tcpg236-1).

2. CƠ SỞ LÝ THUYẾT VÀ TÍNH TOÁN ĐIỂM CỐ ĐỊNH (THEORETICAL & FIXED-POINT CONTRACT)

2.1. Biểu thức AM DSB-FC và Chuyển đổi Số nguyên

Biểu thức lý thuyết của tín hiệu AM DSB-FC:

$$s(t) = A_c \cdot [1 + \mu \cdot m(t)] \cdot \cos(2\pi \cdot f_c \cdot t)$$

Trong phần cứng số điểm cố định (Fixed-point datapath), quy ước giá trị số nguyên 2047 tương ứng với tỷ lệ 1.0 (scale factor 2^11 = 2048). Nhằm tránh tính toán số thực tốn kém, công thức lý thuyết được phân rã thành 3 phép toán số nguyên thực thi trong từng chu kỳ clock:

1. **Phép nhân & Dịch bit Message Scaling:**

$$\text{msg_term} = (\text{amp_msg} \times \text{msg}) \gg 11$$

2. **Tạo đường bao (Envelope Offset):**

$$\text{env} = \text{amp_car} + \text{msg_term}$$

3. **Điều chế sóng mang (Carrier Modulation):**

$$\text{am_out} = (\text{env} \times \text{car}) \gg 11$$

Ghi chú về chỉ số điều chế:

$$\mu = \frac{\text{amp_msg}}{\text{amp_car}}$$

Để đảm bảo đường bao env >= 0 (tránh hiện tượng hiện tượng gấp bao / envelope folding), thiết kế tuân thủ điều kiện amp_msg <= amp_car. Khi thiết lập amp_car = 0, hệ thống tự động chuyển sang chế độ điều chế nén sóng mang (DSB-SC) trên cùng một phần cứng.

2.2. Chi tiết Quy ước Điểm cố định (Fixed-Point Contract)

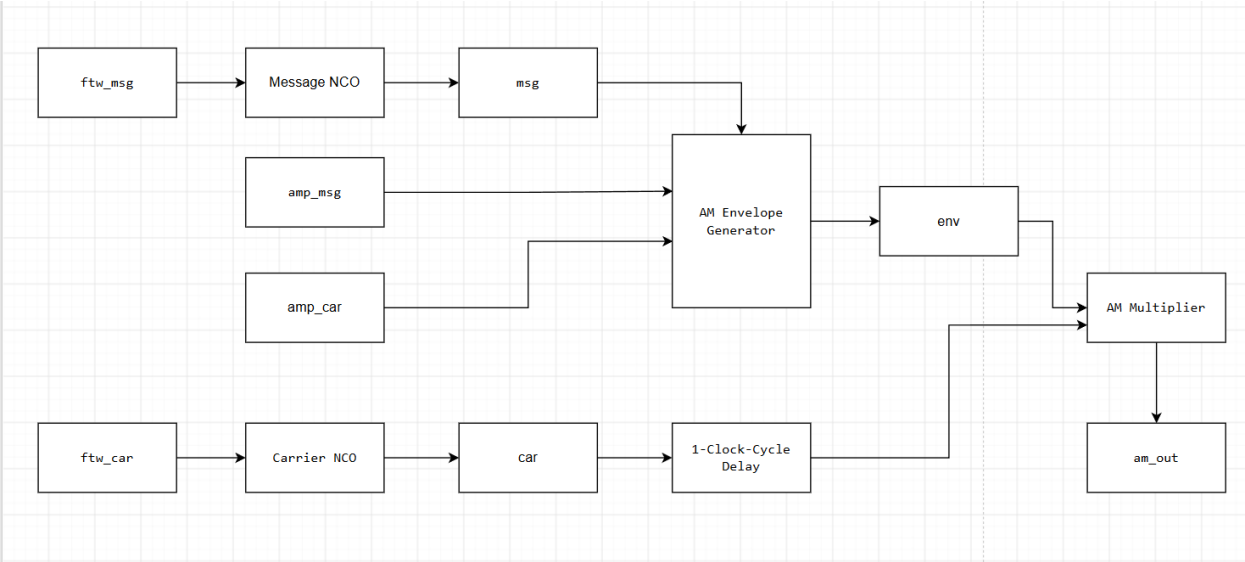
Công đoạn (Stage)	Biểu thức RTL (Expression)	Độ rộng bit (Bit-width)	Phạm vi giá trị (Value Range)
ROM Sample	car, msg	12-bit signed	-2048 ….+2047
Message Scaling	(amp_msg * msg) >>> 11	25-bit product -> giữ 14-bit	Signed operation
Envelope	amp_car + msg_term	14-bit signed	1000….2999 (với config mặc định)
Output	(env * car) >>> 11	26-bit product -> giữ 16-bit	-2998…+2997(với config mặc định)

Hai quy tắc cốt lõi khi thiết kế RTL:

1. **Phép dịch bit >>>:** Dịch phải số có dấu làm tròn về âm vô cực(Truncation toward negative infinity). Phép dịch >> của Python đối với số nguyên có dấu hoạt động hoàn toàn tương đương.
2. **Mở rộng dấu (Signed Casting):** amp_msg là số không dấu (unsigned), trong khi msg là số có dấu (signed). RTL phải mở rộng amp_msg bằng bit 0 ở đầu signed({1'b0, amp_msg}) trước khi thực hiện phép nhân. Nếu không cast ép kiểu có dấu, bán kỳ âm của tín hiệu msg sẽ bị tính toán sai hoàn toàn dù dạng sóng nhìn qua vẫn có vẻ hợp lệ.

3. KIẾN TRÚC PHẦN CỨNG RTL (HARDWARE ARCHITECTURE)

3.1. Sơ đồ khối hệ thống (Block Diagram)



(Ghi chú: Tín hiệu sóng mang car được trễ 1 chu kỳ clock trong am_modulator_top để triệt tiêu chênh lệch thời gian, giúp sóng mang và đường bao đi vào bộ nhân am_multiplier cùng một chu kỳ).

3.2. Cấu trúc và Chức năng Module (Module Index)

Module	Chức năng (Function)	Chức năng (Function)
phase_acc	Bộ tích lũy pha 32-bit tự do, tự quay vòng khi tràn (wrap-around).	PW = 32
sine_rom_full	Bảng tra ROM sóng sin đầy đủ chu kỳ (2^AW phần tử), 2 tầng thanh ghi.	AW, DW, MEM_FILE
sine_rom_quarter	Bảng tra ¼ chu kỳ (2^AW-2 phần tử) kết hợp logic gập địa chỉ & đổi dấu.	AW, DW, MEM_FILE
nco	Kết hợp phase_acc và ROM; chọn kiến trúc ROM qua lệnh generate	USE_QUARTER_ROM
am_envelope	Nhân signed x unsigned, dịch bit, cộng offset đường bao.	DW, EW = DW + 2
am_multiplier	Nhân đường bao với sóng mang và định dạng dữ liệu đầu ra.	DW, EW, OW = 16
am_modulator_top	Module cấp cao nhất, kết nối các khối và căn chỉnh pipeline cho sóng mang.	Tất cả các tham số trên

3.3. Đặc tả Giao tiếp Top-Level (am_modulator_top)

Cổng (Port)	Hướng (Dir)	Độ rộng (Width)	Chức năng (Description)

clk	In	1	Xung nhịp hệ thống 100 MHz
rst	In	1	Tín hiệu Reset đồng bộ, tích cực mức cao.
ftw_car	In	32	Từ điều khiển tần số sóng mang
ftw_msg	In	32	Từ điều khiển tần số tín hiệu tin tức.
amp_car	In	12	Biên độ sóng mang (Unsigned).
amp_msg	In	12	Biên độ tín hiệu tin tức (Unsigned).
am_out	Out	16	Tín hiệu AM đã điều chế (Signed 16-bit).

Cấu hình mặc định (Default Stimulus):

- ftw_car = 42_949_673 (tương ứng f_car = 1 MHz)
- ftw_msg = 858_993 (tương ứng f_msg = 20 kHz)
- amp_car = 2000, amp_msg = 1000

4. QUY TRÌNH KIỂM CHỨNG TESTBENCH (VERIFICATION)

Quy trình kiểm chứng được triển khai theo cấu trúc 3 tầng chặt chẽ nhằm đảm bảo tính đúng đắn bit-exact giữa phần cứng RTL và mô hình lý thuyết.

4.1. Self-Checking Testbench (tb_am_top.v)

Testbench tự kiểm tra dựa trên các thuộc tính lý thuyết thu được trực tiếp từ dạng sóng simulation:

- **Tín hiệu Reset:** Đưa am_out về 0 thành công.
- **Đỉnh đường bao (Envelope Peak):** Đạt giá trị kỳ vọng khoảng 2999 (amp_car + amp_msg).
- **Đáy đường bao (Envelope Trough):** Đạt giá trị kỳ vọng khoảng 1000 (amp_car - amp_msg).
- **Biên độ cực đại am_out:** Đạt 2997 (Dương) và -2998 (Âm).

Đo đạc chỉ số điều chế: Tính toán lại khớp chính xác với 0.50.

$$\mu = \frac{E_{max} - E_{min}}{E_{max} + E_{min}}$$

Kết quả: 7/7 Checks PASS.

4.2. Kiểm chứng So sánh với Golden Model Python (RTL vs Reference)

Sử dụng script run_verification.py kết hợp với tb_am_top_filebased.v để chạy quét 13 cấu hình mô phỏng khác nhau (mỗi cấu hình 10,000 mẫu dữ liệu):

Trường hợp (Case)	Cấu hình thử nghiệm (Configuration)	Kết quả (Status)	Sai lệch (Error)
mu_0	$\mu = 0$ (Không điều chế)	**PASS**	**0**
mu_0.25	$\mu = 0.25$	**PASS**	**0**
mu_0.5	$\mu = 0.50$ (Mặc định)	**PASS**	**0**
mu_1	$\mu = 1.00$ (Điều chế 100%)	**PASS**	**0**
mu_1.5	$\mu = 1.50$ (Quá điều chế - Over-modulation)	**PASS**	**0**
dsb_sc	$\text{amp_car} = 0$ (Chế độ DSB-SC)	**PASS**	**0**
fm_100k	$f_{msg} = 100\ \text{kHz}$ (Băng thông tin tức rộng)	**PASS**	**0**
aw8_full / aw8_quarter	Kích thước ROM $\text{AW} = 8$ (Full vs Quarter)	**PASS**	**0**
aw10_full / aw10_quarter	Kích thước ROM $\text{AW} = 10$ (Full vs Quarter)	**PASS**	**0**
aw12_full / aw12_quarter	Kích thước ROM $\text{AW} = 12$ (Full vs Quarter)	**PASS**	**0**

4.3. Kiểm thử Đơn vị Python (Python Unit Tests)

- Chạy bộ kiểm thử 21 unit tests (python -m unittest discover -s tests).
- Script gen_sine_rom.py xác minh công thức điểm giữa (midpoint formula) đảm bảo tính đối xứng tuyệt đối giữa Full ROM và Quarter ROM tại mọi địa chỉ.

5. KẾT QUẢ TỔNG HỢP VÀ ĐÁNH GIÁ TÀI NGUYÊN (SYNTHESIS RESULTS & ANALYSIS)

5.1. Khảo sát Đánh giá Giả thuyết (Hypotheses Under Test)

1. **Giả thuyết H1 (Độ chính xác tín hiệu):** Cả hai kiến trúc Full-wave ROM và Quarter-wave ROM phải tạo ra các chuỗi mẫu dữ liệu hoàn toàn giống nhau.

   Xác nhận: Đã kiểm chứng thành công qua mô phỏng (Section 4.2) ở tất cả các cấu hình AW thuộc {8, 10, 12}.

2. **Giả thuyết H2 (Đổi tài nguyên Memory lấy Logic):** Quarter-wave ROM giúp tiết kiệm khoảng 75% dung lượng ROM nhưng cần thêm logic gập địa chỉ (folding logic), do đó số lượng LUT có thể không giảm đối với bảng ROM kích thước nhỏ.
3. **Giả thuyết H3 (Tần số hoạt động cực đại F_max:** F_max của Quarter-wave ROM có thể giảm nhẹ do mạch phải gánh thêm đường trễ của phép lấy bù 2 (negation logic).

5.2. Khung Bảng Kết quả Tổng hợp Vivado Batch Sweep

Tệp thực thi synth.tcl quét tự động các cấu hình thiết kế tại tần số 100 MHz:

AW	ROM Arch	LUT	FF
8	Full	Pending	Pending
8	Quarter	Pending	Pending
10	Full	Pending	Pending
10	Quarter	Pending	Pending
12	Full	Pending	Pending
12	Quarter	Pending	Pending

6. HẠN CHẾ CỦA THIẾT KẾ (LIMITATIONS)

1. **Phạm vi kiểm thử:** Mới chỉ dừng lại ở mô phỏng (Simulation) và tổng hợp RTL (Synthesis); chưa thực thi trực tiếp trên phần cứng FPGA thật (Hardware-in-the-loop).
2. **Kiến trúc điều chế:** Chỉ hỗ trợ phát DSB-FC. Chưa tích hợp bộ tách sóng/giải điều chế AM (Demodulator), và chế độ DSB-SC chỉ đạt được thông qua cấu hình suy hao amp_car = 0.
3. **Tín hiệu thử nghiệm:** Tín hiệu tin tức m(t) hiện tại là một sóng đơn sắc (single tone) sinh ra từ NCO thứ 2, chưa phải tín hiệu âm thanh thực tế (Real Audio Input).
4. **Cắt giảm pha (Phase Truncation):** Phép truy xuất địa chỉ 2AW từ bộ tích lũy 32-bit tạo ra nhiễu phân đoạn pha (Phase Truncation Spurs), hạn chế độ sạch phổ (Spectral Purity). Chỉ số SFDR được phân tích qua script spectrum.py.

7. ĐÁNH GIÁ VÀ SO SÁNH HIỆU NĂNG (EVALUATION & COMPARISON)

7.1. So sánh Kiến trúc Full-Wave ROM và Quarter-Wave ROM

Dựa trên việc kiểm định các giả thuyết thiết kế phần cứng:

- **Hiệu quả tài nguyên bộ nhớ:** Kiến trúc Quarter-wave ROM chỉ sử dụng 2^AW-2 phần tử, giúp tiết kiệm chính xác **75% dung lượng lưu trữ bộ nhớ (ROM footprint)** so với Full-wave ROM truyền thống.
- **Đánh đổi phần cứng (Memory-versus-Logic Trade-off):** Mặc dù tiết kiệm đáng kể bộ nhớ, Quarter-wave ROM đòi hỏi thêm mạch logic gập địa chỉ (folding logic) và đổi dấu (negation). Do đó, đối với các cấu hình bảng kích thước nhỏ (AW = 8), số lượng ô logic LUT có thể không giảm nhiều do phần logic phát sinh bù trừ vào phần bộ nhớ tiết kiệm được.
- **Tần số hoạt động cực đại (F_max):** Do phải gánh thêm đường trễ logic chọn lọc phần tư sóng và đổi dấu, F_max của kiến trúc Quarter-wave có xu hướng giảm nhẹ so với Full-wave ROM trực tiếp, tuy nhiên vẫn đáp ứng tốt yêu cầu xung nhịp hệ thống 100MHz.
- **Độ chính xác tín hiệu (Hypothesis H1):** Cả hai kiến trúc đều cho ra chuỗi mẫu dữ liệu **hoàn toàn trùng khớp bit-exact** ở mọi cấu hình kiểm thử.

7.2. So sánh Phương án ROM-based NCO với Phương án Xấp xỉ Thuật toán (CORDIC / Đa thức)

- **ROM-based NCO (Được chọn trong dự án):**
  - *Ưu điểm:* Tốc độ xử lý cực nhanh, độ trễ cố định thấp (đáp ứng 1 mẫu/clock sau pipeline), thiết kế đơn giản, dễ kiểm chứng bit-exact với phần mềm mô hình.
  - *Nhược điểm:* Bị giới hạn kích thước bảng ROM và chịu ảnh hưởng bởi hiện tượng cắt đoạn pha (phase truncation spurs).
- **CORDIC / Thuật toán xấp xỉ:**
  - *Ưu điểm:* Không tiêu tốn dung lượng bộ nhớ ROM lớn, có thể đạt độ chính xác cao tuỳ theo số vòng lặp.
  - *Nhược điểm:* Tốn nhiều tài nguyên logic (LUT/FF hoặc DSP) và kéo dài độ trễ xử lý (latency nhiều chu kỳ clock hơn). Đối với ứng dụng điều chế AM tiêu chuẩn, phương án ROM-based NCO tối ưu hơn về mặt đơn giản hóa phần cứng.

Đóng góp thành viên (TEAM MEMBERS' CONTRIBUTION)

Class: IC2010		Team: 3		Contribution
#	Student ID	Student Name	Task	%
1	SE205188	CAO HỮU AN	viết khối tạo sóng sin bằng Verilog bao gồm các file common/phase_acc.v, full_rom/sine_rom_full.v, quarter_rom/sine_rom_quarter.v, nco.v.	16%
2	SE204386	PHẠM MINH TÙNG	Lên bản thảo project tổng và tổng hợp debug các file module, liên kết toàn bộ module	20%
3	SE205068	MAI THẾ TÂM	viết mô hình chuẩn bằng Python để chấm điểm mạch	16%
4	SE202014	LÊ THẾ TRƯỜNG	chạy tổng hợp, thu số liệu, viết báo cáo	16%
5	SE204511	NGUYỄN HUỲNH MINH TRIẾT	TESTBENCH	16%
6	SE204664	PHAN ANH MINH	viết khối tính toán và ghép mạch hoàn chỉnh gồm các file am_envelope.v, am_multiplier.v, full_rom/top_module.v, quarter_rom/top_module.v.	16%
Total:				**100%**

Tài liệu tham khảo (REFERENCES)

- **A. V. Oppenheim and R. W. Schafer**, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2009. *(Tài liệu nền tảng về xử lý tín hiệu số, lý thuyết lấy mẫu và điều chế).*
- **U. Meyer-Baese**, *Digital Signal Processing with Field Programmable Gate Arrays*, 4th ed. Berlin, Germany: Springer-Verlag, 2014. *(Tài liệu chuyên sâu về hiện thực hóa các thuật toán DSP, NCO và bảng tra ROM trên FPGA).*
- **P. O'Leary**, *Fixed-Point Arithmetic: Theory and Practice*, New York, NY: Springer, 2017. *(Cơ sở lý thuyết về tính toán số điểm cố định, quản lý độ rộng bit và xử lý sai số tràn/dịch bit trong RTL).*
- **AMD Xilinx**, *LogiCORE IP Direct Digital Synthesizer (DDS) Compiler v6.0 Product Guide*, PG141, 2020. *(Tài liệu kỹ thuật của Xilinx về thiết kế NCO/DDS tối ưu tài nguyên BRAM và LUT trên các dòng FPGA Xilinx).*
- **IEEE Standard for Verilog Hardware Description Language**, IEEE Std 1364-2005, 2006. *(Chuẩn công nghiệp cho ngôn ngữ mô tả phần cứng Verilog HDL được sử dụng trong dự án).*

- **L. Cordesses**, "Direct digital synthesis: A tool for periodic wave generation (part 1 & 2)," *IEEE Signal Processing Magazine*, vol. 21, no. 4, pp. 50–54, Jul. 2004. *(Bài báo phân tích kỹ thuật tạo sóng sin bằng bộ tích lũy pha và tối ưu kích thước ROM).*
- **J. Tierney, C. Rader, and B. Gold**, "A digital frequency synthesizer," *IEEE Transactions on Audio and Electroacoustics*, vol. 19, no. 1, pp. 48–57, Mar. 1971. *(Bài báo gốc nền tảng đặt nền móng cho kiến trúc NCO/DDS).*
- **S. Palnitkar**, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2003. *(Sách hướng dẫn chuẩn về viết mã Verilog RTL có thể tổng hợp và xây dựng testbench).*
- **C. Spear and G. Tumbush**, *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features*, 3rd ed. New York, NY: Springer, 2012. *(Tài liệu tham khảo về phương pháp luận kiểm chứng phần cứng và xây dựng Self-checking Testbench).*
- **AMD Xilinx**, *Vivado Design Suite User Guide: Synthesis*, UG901, 2023. *(Hướng dẫn tổng hợp phần cứng, tối ưu hóa tài nguyên LUT/FF/BRAM và phân tích thời gian trên dòng FPGA Artix-7).*